

ITC MP, DS n°1

Modélisation numérique d'un matériau magnétique

Ce sujet est un extrait modifié du sujet des Mines 2022.

Certains matériaux particuliers peuvent acquérir des états magnétiques qualifiés de paramagnétique et ferromagnétique. Le matériau est dit **paramagnétique** lorsqu'il ne possède pas d'aimantation spontanée, mais acquiert une aimantation sous l'effet d'un champ magnétique extérieur. Il est dit **ferromagnétique** lorsqu'il possède une aimantation même en l'absence de champ magnétique extérieur. Dans ces matériaux, la température T joue un rôle crucial : si T est supérieure à une température particulière T_C , nommée température de Curie, le matériau adopte un état paramagnétique. Dans le cas contraire ($T < T_C$), il adopte un état ferromagnétique. C'est par exemple le cas du fer, pour lequel la transition entre les deux états se produit à $T_C = 1043$ kelvin.

Dans un matériau magnétique, les divers éléments magnétiques (électrons, atomes) possédant un moment magnétique créent une aimantation moyenne à l'intérieur du matériau. Nous admettrons les principaux résultats de la théorie du paramagnétisme.

Ce sujet est constitué de 3 parties indépendantes. Dans la première, on cherche à obtenir l'aimantation moyenne du matériau en fonction de la température à partir d'une formule théorique connue. Dans la seconde, on recherche dans une base de données les propriétés de matériaux magnétiques. Dans la troisième, on cherche à développer une modélisation microscopique d'un matériau magnétique à deux dimensions pour retrouver ce comportement (modèle d'Ising).

Les candidats sont fortement incités à expliciter brièvement leurs programmes à l'aide de quelques commentaires bien placés, et à soigner la présentation de leur code. En particulier :

- les noms de variables doivent être explicites quand leur usage n'est pas immédiatement clair.
- l'indentation de votre programme doit être spécifiée (par exemple, par des lignes).

L'utilisation du module numpy n'est pas autorisée.

I) Transition paramagnétique/ferromagnétique sans champ magnétique extérieur

La théorie des matériaux indique que, dans un matériau ferromagnétique, l'aimantation volumique moyenne du matériau est donnée par :

$$M = N\mu \tanh\left(\frac{\mu B}{k_B T}\right) \quad (1)$$

où N est le nombre d'atomes par unité de volume, B est la valeur du champ magnétique à l'intérieur du matériau, μ le moment magnétique des atomes ou des ions, k_B la constante de Boltzmann, T la température et $\tanh : x \mapsto \frac{e^x - e^{-x}}{e^x + e^{-x}}$ la fonction tangente hyperbolique.

On considère une situation sans champ magnétique extérieur. Le champ magnétique local à l'intérieur du matériau ferromagnétique est donc celui créé par le matériau lui-même. On admet que ce champ magnétique est proportionnel à l'aimantation moyenne dans le matériau ($B = \mu M$), et on obtient alors : $M = N\mu \tanh\left(\frac{\mu \lambda M}{k_B T}\right)$.

En introduisant l'aimantation réduite $m = \frac{M}{N\mu}$ et la température réduite $t = \frac{k_B T}{N\mu^2 \lambda} = \frac{T}{T_C}$, l'équation 1 devient :

$$m = \tanh\left(\frac{m}{t}\right) \quad (2)$$

Cette équation d'inconnue m ne possède pas de solution analytique : si on veut connaître une approximation de l'aimantation moyenne dans le matériau, il est donc nécessaire de la résoudre numériquement par une méthode de recherche de zéro.

Pour importer une ou plusieurs fonction d'un module python, on écrit :

```
from module import fonction1, fonction2
```

1. Écrire les instructions nécessaires pour importer exclusivement les fonctions exponentielle (exp) et tangente hyperbolique (tanh) du module math. Ces fonctions seront ainsi utilisables dans tous les programmes que vous écrirez ultérieurement.
2. A partir de l'équation 2, indiquer une équation $f(x, t) = 0$, d'inconnue x que l'on doit résoudre et écrire en Python la définition de la fonction f correspondante (paramètres x et t , valeur renvoyée $f(x, t)$).
3. Écrire une fonction recherche_lineaire(f , t , a , b , eps) qui calcule par une recherche linéaire une valeur approchée à eps près du zéro d'une fonction $f(m, t)$ de variable m et de paramètre t fixé sur un intervalle $[a, b]$. On supposera pour simplifier que la fonction dont on recherche le zéro est continue et s'annule une fois et une seule sur l'intervalle $[a, b]$.

On pourra, par exemple, tester toutes les valeurs de l'intervalle à eps près : $a, a+eps, a+2*eps, \dots, b$ et prendre celle dont l'image par f est la plus proche de 0.

4. On souhaite améliorer la fonction recherche_lineaire. Pour cela, on va utiliser la dichotomie, avec le code suivant :

```
def recherche_dicho(f, t, a, b, eps):
    """
    Prend en argument :
    - une fonction f, prenant deux arguments flottants m et t, et renvoyant un
    flottant. m -> f(m, t) est continue et s'annule une fois sur [a, b].
    - un flottant t
    - un intervalle [a, b]
    - une précision eps > 0

    La fonction effectue une recherche dichotomique et renvoie, à eps près, un m dans
    [a, b] tel que f(m, t) = 0.
    """
    while b - a > eps:
        milieu = (a + b) / 2
        #test si f(milieu) et f(a) ont le même signe
        if f(a, t) * f(milieu, t) >= 0.0:
            #dans ce cas, la fonction s'annule dans l'intervalle [milieu, b]
            a = milieu
        else:
            #dans le cas inverse, la fonction s'annule dans [a, milieu]
            b = milieu
    #Ici, n'importe quelle valeur de l'intervalle convient
    #On prend le centre
    return (a + b) / 2
```

- a) En supposant f , t et eps fixés, donnez un test au cas limite pour recherche_dicho.
- b) Montrez la terminaison de la fonction.
- c) Montrez la correction partielle de la fonction.

Une étude mathématique simple permet de prouver que l'équation $m = \tanh\left(\frac{m}{t}\right)$ n'a de solution $m > 0$ que pour $0 < t < 1$, ce qui revient à dire que le matériau ne possède une aimantation non nulle que pour une température inférieure à la température de Curie. On admet ainsi que si $t \geq 1$ alors $m = 0$. De plus, pour $t < 1$, on admet que la solution $m = 0$ ne doit pas être prise en compte car elle correspond à une solution instable.

5. En utilisant la fonction `recherche_dicho`, écrire une fonction `construction_liste_m(t1, t2)` qui construit et retourne une liste de 500 solutions de l'équation (1), pour t variant linéairement de t_1 à t_2 (bornes incluses). On cherchera les valeurs de m à 10^{-6} près avec un intervalle de recherche initial $m \in [0.001, 1]$.

En traçant l'aimantation m en fonction de la température t , on obtient le graphe de la figure 1 permettant de visualiser les domaines ferromagnétique ($t < 1$) et paramagnétique ($t \geq 1$).

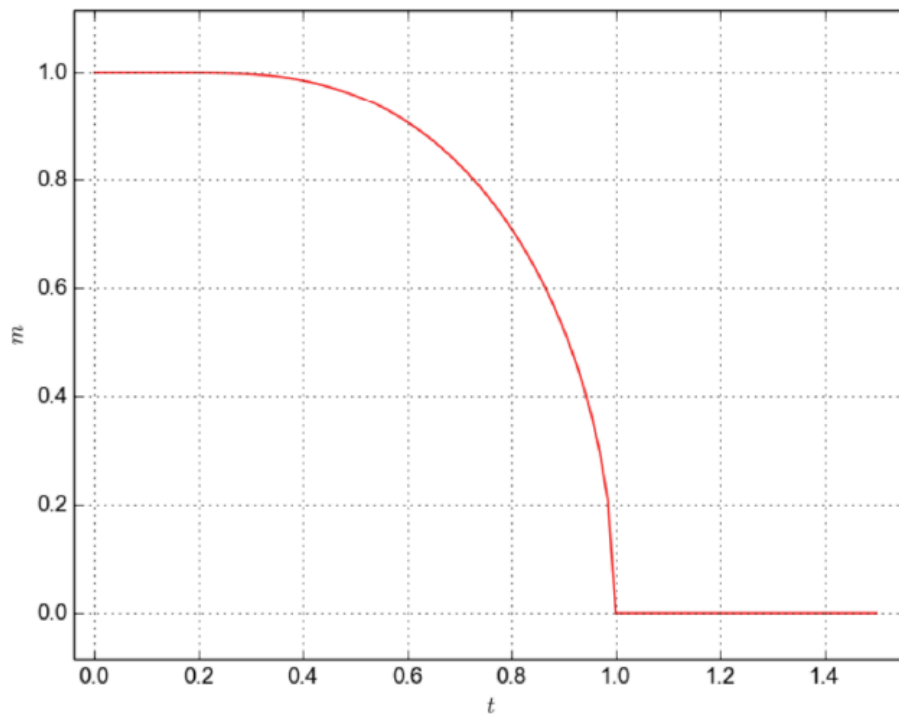


Figure 1: aimantation réduite m en fonction de la température réduite t

II) Recherche dans une base de données de matériaux magnétiques

Il existe des bases de données contenant les propriétés de nombreux matériaux, dont des propriétés magnétiques. Dans cette partie, on donne un modèle simplifié d'une telle base, et on souhaite effectuer quelques requêtes sur celle-ci.

La base de données possède la structure suivante :

- La table `matériaux` contient un champ `id_materiau`, clé primaire de la table de valeur entière, un champ `nom` de type chaîne de caractères pour le nom du matériau et un champ `t_curie` de valeur entière pour la température de Curie du matériau en kelvin.

id_materiau	nom	t_curie
4534	cobalt	1 388
1254	dioxyde de chrome	386
8713	nickel	627
8284	YIG	560
...	...	...

- La table fournisseurs, contenant un champ id_fournisseur, clé primaire de type entier qui précise le code de chaque fournisseur, et un champ nom_fournisseur de type chaîne de caractères pour le nom du fournisseur.

id_fournisseur	nom_fournisseur
145	Worldwide Materials
13	Materials Company
...	...

- La table prix qui contient un champ id_prix, clef primaire de type entier, un champ id_mat qui est une clé étrangère pointant vers le champ id_materiau de la table matériaux, un champ id_four qui est une clé étrangère pointant sur le champ id_fournisseur de la table fournisseurs, et un champ prix_kg de type flottant qui précise le prix au kg que ce fournisseur propose pour ce matériau, en euros. Un fournisseur qui ne propose pas un matériau donné n'a pas d'entrée correspondante dans cette table.

id_prix	id_mat	id_four	prix_kg
1	4567	145	50.40
2	8671	13	1357.30
3	1763	145	52.75
...	...	...	...

Les requêtes demandées dans cette partie sont à écrire en langage SQL.

6. Écrire une requête permettant d'obtenir le nom de tous les matériaux qui ont une température de Curie strictement inférieure à 500 kelvins.

Un client potentiel souhaite acheter 4,5 kilogrammes de nickel et sélectionner le fournisseur le moins cher.

7. Écrire une requête permettant d'obtenir l'identifiant du nickel.

Par la suite, on pourra utiliser cet identifiant, 8713, dans les requêtes.

8. Écrire une requête permettant d'obtenir les noms de tous les fournisseurs proposant du nickel et le prix proposé par chacun pour 4,5 kilogrammes de nickel.
9. Modifier ou compléter la requête précédente afin d'obtenir le nom du fournisseur de nickel le moins cher ainsi que le prix à payer chez ce fournisseur pour ces 4,5 kilogrammes de nickel.
10. Écrire une requête permettant d'obtenir le nom de tous les matériaux et le prix moyen pour un kilogramme de chacun de ces matériaux (la moyenne étant calculée pour tous les fournisseurs proposant ce matériau), en se limitant aux prix moyens strictement inférieurs à 50 euros par kilogramme.

III) Modèle microscopique d'un matériau magnétique

Pour étudier l'effet du champ magnétique sur un matériau magnétique, on adopte une modélisation microscopique. On modélise les atomes par des sites portant chacun une grandeur physique, nommée *spin*, dont il n'est pas nécessaire de connaître les propriétés.

L'échantillon modélisé est une zone carrée à deux dimensions possédant h *spins* régulièrement répartis dans chaque direction, donc formant une grille carrée de $n = h^2$ *spins*. Chaque *spin* ne possède que deux états *down* ou *up*, ce que l'on modélise par une variable $s_i \in \{-1, +1\}$.

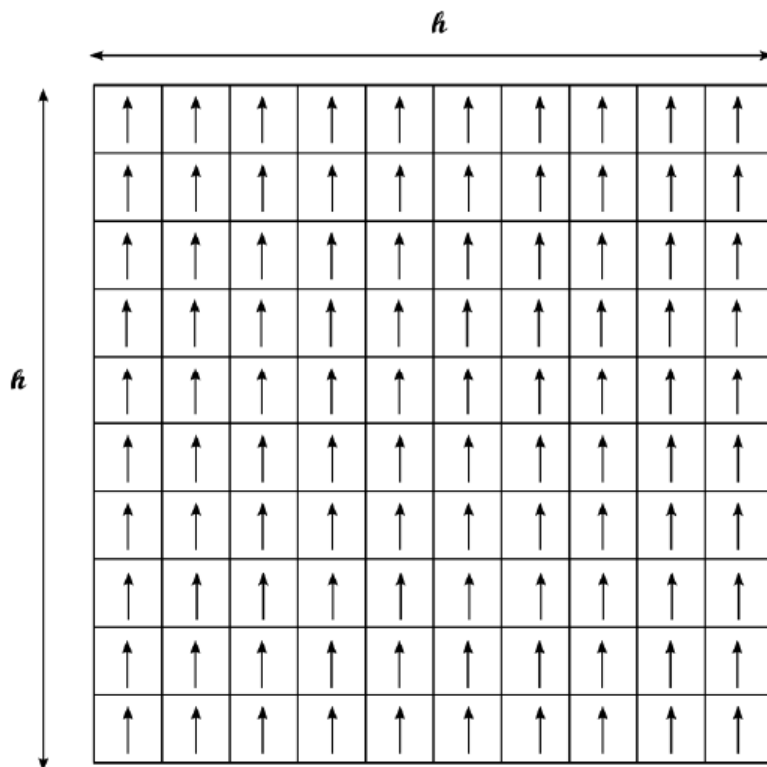


Figure 2: Modèle des *spins* dans un matériau ferromagnétique

Pour implémenter cette configuration de *spins* décrivant l'état microscopique du matériau, on choisit de travailler sur une liste s , contenant n entiers, chacun valant -1 ou 1 . On notera le choix d'implémentation adoptée, qui impose de travailler sur une simple liste de n éléments pour modéliser une grille de taille $n = h * h$, dans l'ordre suivant : première ligne puis deuxième ligne , etc.

Pour rappel, les listes Python sont des objets contenant une liste de valeurs. On accède à la valeur numéro i de s avec $s[i]$. Les valeurs sont numérotées à parti de zéro.

Pour créer une nouvelle liste vide, on peut écrire $s = []$. Pour y ajouter un élément, on écrit $s.append(-1)$

Un domaine d'aimantation uniforme (cf. figure 2) sera donc représenté par une liste contenant n fois 1 ($[1, 1, 1, \dots, 1]$). Le début du programme, outre les imports de module Python déjà réalisés à la question 1, est défini par :

```
h = 100
n = h ** 2
```

ce qui définit deux variables globales utilisables dans tout le programme.

11. Écrire une fonction `initialisation()` renvoyant une liste d'initialisation des domaines contenant n spins de valeur 1 comme sur la figure 2.

L'antiferromagnétisme est une propriété de certains milieux magnétiques. Contrairement aux matériaux ferromagnétiques, dans les matériaux antiferromagnétiques, l'interaction d'échange entre les atomes voisins conduit à un alignement antiparallèle des moments magnétiques atomiques (cf. figure 3). L'aimantation totale du matériau est alors nulle (on se limite au cas où h est pair).

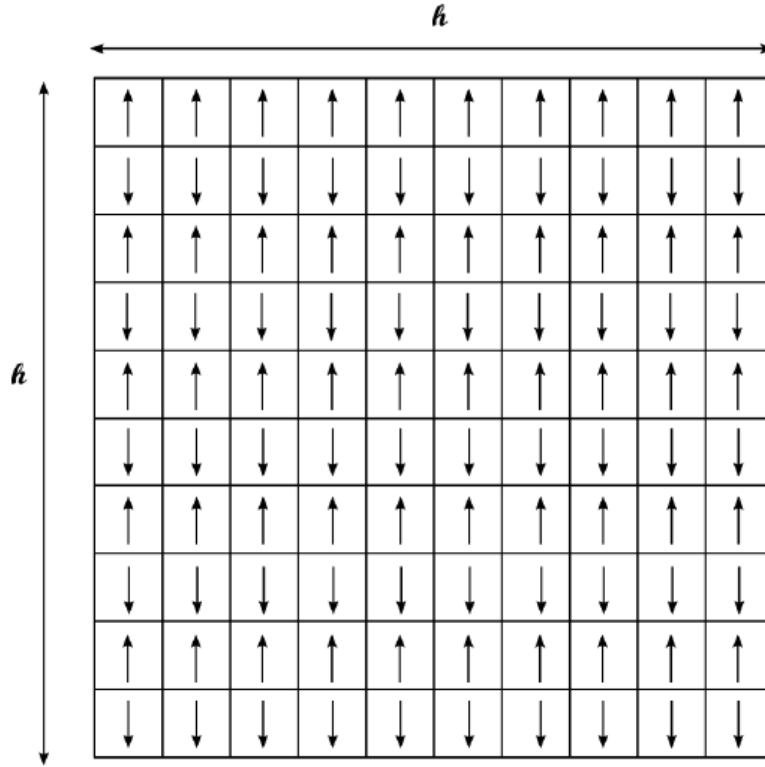


Figure 3: Modèle des spins dans un matériau antiferromagnétique

12. Écrire une fonction `initialisation_anti()` renvoyant une liste `s` d'initialisation des domaines contenant h spins en largeur et h en hauteur en alternant les 1 et -1 comme sur la figure 3.

Dans la modélisation adoptée (sans champ magnétique extérieur), l'énergie d'une configuration, définie par l'ensemble des valeurs de tous les spins, est donnée par :

$$E = -\frac{J}{2} \sum_i \sum_{j \in V_i} s_i s_j \quad (3)$$

On suppose que seuls les quatre spins situés juste au dessus, en dessous, à gauche et à droite de s_i sont capables d'interagir avec lui.

J est nommée intégrale d'échange et modélise l'interaction entre deux spins voisins. Pour simplifier, on considérera dans les programmes que $J = 1$.

Malgré le caractère fini de l'échantillon, on peut utiliser une modélisation très utile pour faire comme s'il était infini en utilisant les conditions aux limites périodiques. Lorsque l'on considère un spin dans la colonne située la plus à droite (resp. gauche), il ne possède pas de plus proche voisin à droite (resp. gauche) : on convient de lui en affecter un, qui sera situé sur la même ligne complètement à gauche (resp. droite) de l'échantillon. De même, le plus proche voisin manquant

d'un *spin* situé sur la première (resp. dernière) ligne sera situé sur la dernière (resp. première) ligne de l'échantillon (voir la figure 4)

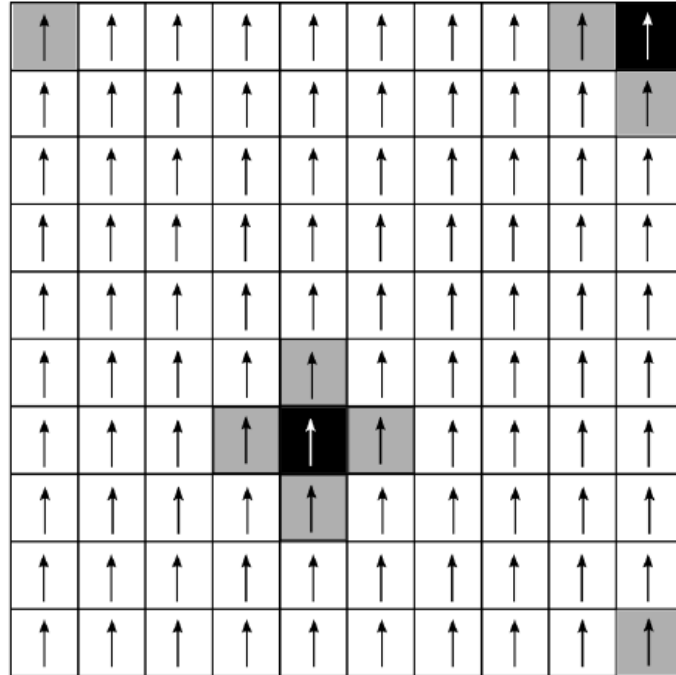


Figure 4: Voisinage d'un *spin* (les voisins des *spins* sur les cases noires sont indiqués en gris)

13. Définir une fonction `liste_voisins(i)` qui renvoie la liste des indices des plus proches voisins du *spin* s_i d'indice i dans la liste s (dans l'ordre gauche, droite, dessous, dessus). On pourra utilement utiliser les opérations `%` et `//` de Python, qui renvoient le reste et le quotient de la division euclidienne.
14. Définir la fonction `energie(s)` qui calcule l'énergie d'une configuration s donnée (cf. équation 3).